

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ФАКУЛЬТЕТ ПРИКЛАДНОЙ МАТЕМАТИКИ И ИНФОРМАТИКИ
Кафедра методов оптимального управления

СИНТЕЗ ОПТИМАЛЬНЫХ СИСТЕМ НА ОСНОВЕ МЕТОДОВ КЛАССИФИКАЦИИ

Курсовая работа

Головач Алексей Викторович
студента 3 курса специальности
«прикладная
математика»

Научный руководитель:
старший преподаватель
М.А. Готовец

Минск, 2026

ОГЛАВЛЕНИЕ

	С.
ВВЕДЕНИЕ	3
ГЛАВА 1 ОСНОВНЫЕ ПОНЯТИЯ И МЕТОДОЛОГИЯ . . .	5
1.1 Задачи оптимального управления линейными системами	5
1.2 Структура оптимальной обратной связи	6
1.3 Применение методов классификации для синтеза субоптималь- ной обратной связи	7
1.4 Алгоритм реализации	9
ГЛАВА 2 ЧИСЛЕННЫЕ ЭКСПЕРИМЕНТЫ	10
2.1 Описание экспериментальных данных.	10
2.2 Процесс классификации и структура данных	10
2.3 Синтез субоптимальной обратной связи методом опорных векторов.	11
2.4 Результаты визуализации	13
2.5 Характеристики и качество аппроксимации.	13
2.6 Вычислительная сложность и время выполнения	14
ЗАКЛЮЧЕНИЕ	16
ПРИЛОЖЕНИЕ А	18

ВВЕДЕНИЕ

Теория оптимального управления является одной из фундаментальных областей математики и автоматики, находящей широкое применение в инженерных, экономических и биологических системах. Однако практическая реализация оптимальных обратных связей в реальном времени встречает серьёзные трудности, обусловленные высокой вычислительной сложностью. Классические подходы, основанные на принципе максимума Понтрягина и динамическом программировании, ограничены системами малой размерности. В то же время, методы машинного обучения, в частности метод опорных векторов, интенсивно развиваются и показывают высокую эффективность при решении задач классификации. Синтез этих двух подходов — теории оптимального управления и методов данных — открывает новые возможности для построения субоптимальных обратных связей, пригодных для практической реализации в системах высокой размерности.

В научной литературе достаточно хорошо изучены классические методы синтеза оптимальных систем управления. Работы Л.С. Понтрягина, Р. Беллмана и их последователей заложили фундамент современной теории. В последние два десятилетия активно развивается направление управления в режиме реального времени, позволяющее реализовать квазиоптимальное управление путём многократного решения задач оптимального программного управления в процессе эксплуатации системы.

Цель работы: Разработать и реализовать численный метод построения субоптимальных обратных связей в линейных задачах оптимального управления, основанный на применении метода опорных векторов к данным, накопленным в процессе управления объектом.

Задачи исследования: Исследовать структурные свойства оптимальной обратной связи в линейных задачах оптимального управления и обосновать её кусочно-аффинность, разработать алгоритм поиска соседних критических областей на основе анализа расстояний между состояниями различных классов, построить методику применения линейного метода опорных векторов для аппроксимации границ между критическими областями, реализовать полный программный комплекс на языке MATLAB, включающий сбор данных, классификацию, обучение моделей и визуализацию результатов, провести численные эксперименты на примере конкретной системы и оценить качество полученной субоптимальной обратной связи.

В первой главе формулируются основные понятия теории оптимального управления, рассматриваются задачи линейного оптимального управления и обосновывается кусочно-аффинная структура оптимальной обратной связи. Приводятся основные идеи применения методов классификации для синтеза субоптимальной обратной связи.

Во второй главе описывается алгоритм построения субоптимальной обратной связи: поиск соседних критических областей, применение метода опорных векторов для построения разделяющих гиперплоскостей и аппроксимация управления в каждой области. Излагаются численные эксперименты на примере двумерной линейной системы. Демонстрируется процесс выделения критических областей, построение границ между ними.

В заключении приводятся краткие выводы по результатам проведённой работы, обсуждаются полученные результаты и даются рекомендации по перспективам дальнейших исследований, включая обобщение метода на системы высокой размерности и применение нелинейных классификаторов.

В приложении представлена реализация разработанного метода на языке MATLAB. Приводятся комментарии к алгоритму, описывается структура программного кода, включая процедуры сбора данных, обучения моделей SVM.

ГЛАВА 1

ОСНОВНЫЕ ПОНЯТИЯ И МЕТОДОЛОГИЯ

В данной главе формулируются основные понятия и методология, используемые при синтезе субоптимальных обратных связей методом опорных векторов. Приводятся основы теории оптимального управления и задачи линейного оптимального управления, для которых разрабатывается предлагаемый подход. Рассматривается структура оптимальной обратной связи как кусочно-аффинной функции состояния и обосновывается возможность применения методов классификации данных для её аппроксимации. Кратко описаны основные идеи метода опорных векторов и алгоритм поиска соседних критических областей.

1.1 Задачи оптимального управления линейными системами

Рассмотрим линейную управляемую систему на промежутке времени $T = [t_0, t_f]$:

$$\dot{x} = A(t)x + b(t)u, \quad x(t_0) = x_0, \quad (1.1)$$

где $x = x(t) \in \mathbb{R}^n$ — состояние системы в момент времени t ; $u = u(t) \in \mathbb{R}$ — скалярное управляющее воздействие; $A(t) \in \mathbb{R}^{n \times n}$, $b(t) \in \mathbb{R}^n$ — кусочно-непрерывные матрица и вектор; x_0 — известное начальное состояние.

Управления выбираются из класса дискретных управлений:

$$u(t) \equiv u(s), \quad t \in [s, s + h), \quad s \in T_h = \{t_0, t_0 + h, \dots, t_f - h\},$$

где $h = (t_f - t_0)/N$ — период квантования, $N > 1$ — натуральное число.

Рассматриваются две задачи оптимального управления:

Определение 1.1 Терминальная задача:

$$c'x(t_f) \rightarrow \max, \quad (1.2)$$

при ограничениях:

$$\dot{x} = A(t)x + b(t)u, \quad x(t_0) = x_0, \quad x(t_f) \in X_f, \quad |u(t)| \leq 1, \quad t \in T,$$

где $c \in \mathbb{R}^n$; $X_f = \{x \in \mathbb{R}^n : Hx \leq g\}$, $H \in \mathbb{R}^{m \times n}$, $g \in \mathbb{R}^m$ — терминальное множество.

Определение 1.2 Задача минимизации полного импульса управления:

$$\int_T |u(t)| dt \rightarrow \min, \quad (1.3)$$

при ограничениях, аналогичных терминальной задаче.

Определение 1.3 Управляющее воздействие $u(t) \in U$, $t \in T$, называется допустимой программой, если соответствующая ему траектория $x(t)$, $t \in T$, удовлетворяет условию $x(t_f) \in X_f$.

Определение 1.4 Программа $u^0(t)$, $t \in T$, называется оптимальной программой, если на соответствующей ей траектории $x^0(t)$ выполняется равенство

$$c'x^0(t_f) = \max_u c'x(t_f)$$

или $\int_T |u^0(t)| dt = \min_u \int_T |u(t)| dt$ (в зависимости от постановки задачи), где максимум/минимум ищется среди всех допустимых программ.

1.2 Структура оптимальной обратной связи

Для реализации обратной связи по состоянию задача погружается в семейство позиционных задач. Для каждой позиции (τ, x_τ) , где $\tau \in T_h$ и x_τ — измеренное состояние в момент τ , решается задача оптимального программного управления на оставшемся горизонте $[\tau, t_f]$.

Определение 1.5 Функция

$$u^0(\tau, x) = u^0(\tau \mid \tau, x), \quad x \in X_\tau, \quad \tau \in T_h,$$

где $u^0(t \mid \tau, x)$ — первый момент оптимальной программы для позиции (τ, x) , называется оптимальной обратной связью по состоянию.

Ключевым результатом теории является следующий факт.

Теорема 1.1 (О структуре оптимальной обратной связи) В задачах оптимального управления вида (1.2) и (1.3) оптимальная обратная связь $u^0(\tau, x)$ является кусочно-аффинной функцией состояния x для

каждого фиксированного τ . Множество допустимых состояний X_τ разбивается на многогранные критические области K_τ^l , в каждой из которых обратная связь линейна:

$$u^0(\tau, x) = (k_\tau^l)^\top x + a_\tau^l, \quad x \in K_\tau^l, \quad l \in L_\tau,$$

где $k_\tau^l \in \mathbb{R}^n$, $a_\tau^l \in \mathbb{R}$.

Это свойство кусочной аффинности следует из теории многопараметрического линейного программирования и является основой для применения методов классификации данных при синтезе субоптимальной обратной связи.

1.3 Применение методов классификации для синтеза субоптимальной обратной связи

Классический подход к построению оптимальной обратной связи посредством решения многопараметрических задач ограничен высокой вычислительной сложностью и применим только для систем малой размерности. Альтернативный подход предполагает использование данных, генерируемых системой управления в реальном времени.

При управлении объектом в режиме реального времени накапливается большой объём информации о позициях (состояниях) и соответствующих им оптимальных управлениях. Идея состоит в том, чтобы использовать эти данные для обучения субоптимальной обратной связи методами машинного обучения, в частности методом опорных векторов.

Для каждого момента времени $\tau \in T_h$ формируется выборка точек и их классификация:

$$\{(x_\tau^i, y_\tau^i)\}_{i=1}^{M_\tau},$$

где x_τ^i — состояние системы, а y_τ^i — метка класса (номер критической области), которой принадлежит точка x_τ^i .

Метка класса определяется на основе структуры оптимальной программы: две точки относятся к одному классу (одной критической области), если соответствующие им оптимальные программы имеют одинаковую структуру (активные ограничения, точки переключения управления).

Метод опорных векторов (Support Vector Machine, SVM) является одним из наиболее эффективных методов линейной бинарной классификации. Для пары классов l и j строится разделяющая гиперплоскость:

$$\omega'x + b = 0,$$

которая максимизирует зазор между двумя классами. Задача формулируется как задача квадратичного программирования:

$$\min_{\omega, b} \|\omega\|^2, \quad \text{при условии} \quad y_i(\omega'x_i + b) \geq 1, \quad i = 1, \dots, M,$$

где $y_i \in \{-1, +1\}$ — метки классов.

Перед применением SVM необходимо определить, какие классы считаются соседними (геометрически смежными). Две области считаются соседними, если существует достаточное количество пар близких точек из этих областей.

Определение 1.6 Классы l и j называются соседними в момент τ , если число пар точек (x_τ^p, x_τ^q) , где первая принадлежит классу l , а вторая — классу j , таких что расстояние между ними не превышает заданный радиус R , не менее заданного порога (например, 5).

Для соседних классов строится одна разделяющая прямая (в двумерном случае) или гиперплоскость (в общем случае), которая служит границей между соответствующими критическими областями.

После построения гиперплоскостей для всех пар соседних классов получается разбиение пространства состояний на многогранные области:

$$\tilde{K}_\tau^l = \{x \in \mathbb{R}^n : \tilde{H}_\tau^l x \leq \tilde{g}_\tau^l\}.$$

В каждой области значение субоптимальной обратной связи задаётся либо константой (если все точки области имеют одно управление), либо вычисляется как линейная функция:

$$\tilde{u}^0(\tau, x) = (\tilde{k}_\tau^l)^\top x + \tilde{a}_\tau^l, \quad x \in \tilde{K}_\tau^l.$$

Для предотвращения нарушения ограничений используется функция

насыщения:

$$\tilde{u}^0(\tau, x) = \text{sat}\left((\tilde{k}_\tau^l)^\top x + \tilde{a}_\tau^l\right) = \begin{cases} 1, & \text{если } (\tilde{k}_\tau^l)^\top x + \tilde{a}_\tau^l > 1, \\ (\tilde{k}_\tau^l)^\top x + \tilde{a}_\tau^l, & \text{если } |(\tilde{k}_\tau^l)^\top x + \tilde{a}_\tau^l| \leq 1, \\ -1, & \text{если } (\tilde{k}_\tau^l)^\top x + \tilde{a}_\tau^l < -1. \end{cases}$$

1.4 Алгоритм реализации

Полный алгоритм построения субоптимальной обратной связи состоит из следующих шагов: решение задач оптимального управления для совокупности начальных состояний либо в режиме реального времени, либо в режиме моделирования, для каждого момента времени определение класса каждого состояния на основе структуры оптимальной программы, для каждого класса определение списка соседних классов на основе расстояния между точками, для каждой пары соседних классов построение разделяющей гиперплоскости методом опорных векторов, построение графиков с отображением точек данных и границ критических областей.

В данной главе рассмотрены основные понятия теории оптимального управления, необходимые для разработки предлагаемого метода. Показано, что оптимальная обратная связь в линейных задачах оптимального управления является кусочно-аффинной функцией состояния, что делает возможным применение методов линейной классификации для её аппроксимации. Приведено описание алгоритма поиска соседних критических областей и построения разделяющих гиперплоскостей методом опорных векторов. Обосновано, что использование данных, генерируемых системой управления, позволяет эффективно синтезировать субоптимальные обратные связи без явного решения многопараметрических задач оптимизации, что особенно важно для систем высокой размерности.

ГЛАВА 2

ЧИСЛЕННЫЕ ЭКСПЕРИМЕНТЫ

В данной главе представлены результаты применения разработанного метода синтеза субоптимальных обратных связей на основе готовой выборки данных. Демонстрируется процесс классификации состояний по критическим областям, выделение соседних классов, построение разделяющих гиперплоскостей методом опорных векторов и визуализация полученных результатов.

2.1 Описание экспериментальных данных

Исследование проводилось на основе готовой выборки данных, содержащей информацию о состояниях двумерной динамической системы: координата x состояния системы, диапазон значений $[-2, 2]$, координата y состояния системы, диапазон значений $[-1.5, 1.5]$, номер класса, от 1 до 12.

Выборка содержит всего 1442 точек состояний (x_i, y_i) , каждой из которых предварительно назначен класс (номер критической области) на основе структуры оптимальной программы управления.

Пространство состояний охватывает область, в которой система может находиться под управлением в соответствии с заданными ограничениями. Геометрическое расположение классов отражает структуру критических областей оптимальной обратной связи для исходной задачи управления.

2.2 Процесс классификации и структура данных

Для выявления пар классов, между которыми необходимо строить разделяющие гиперплоскости, был применён алгоритм поиска соседства на основе анализа расстояний между точками.

Два класса считались соседними, если между ними существовало по крайней мере 5 пар точек (одна из первого класса, одна из второго) на

расстоянии не более $R = 0.1$ в евклидовой метрике:

$$\rho((x_i, y_i), (x_j, y_j)) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \leq 0.1.$$

Результаты анализа показали следующие соседские связи:

Класс 1 соседний с классами: 3, 6, 8, 10 (4 соседа).

Класс 2 соседний с классами: 4, 8, 11, 12 (4 соседа).

Класс 3 соседний с классами: 1, 10 (2 соседа).

Класс 4 соседний с классами: 2, 8 (2 соседа).

Класс 5 соседний с классами: 9, 11 (2 соседа).

Класс 6 соседний с классами: 1, 8 (2 соседа).

Класс 7 соседний с классами: 9, 10 (2 соседа).

Класс 8 соседний с классами: 1, 2, 4, 6 (4 соседа).

Класс 9 соседний с классами: 5, 7, 10, 11 (4 соседа).

Класс 10 соседний с классами: 1, 3, 7, 9 (4 соседа).

Класс 11 соседний с классами: 2, 5, 9, 12 (4 соседа).

Класс 12 соседний с классами: 2, 11 (2 соседа).

Всего было выявлено 18 пар соседних классов, для каждой из которых была построена разделяющая гиперплоскость. Сложная структура соседства указывает на то, что пространство состояний разбивается на множество взаимодействующих критических областей.

2.3 Синтез субоптимальной обратной связи методом опорных векторов

Для каждой пары соседних классов l и j был построен линейный разделяющий классификатор методом опорных векторов в форме гиперплоскости

$$\omega^\top x + b = 0,$$

где $\omega = [w_1, w_2]^\top \in \mathbb{R}^2$ — вектор нормали, $b \in \mathbb{R}$ — смещение. Построение выполнялось в соответствии с постановкой линейного SVM: выбирается гиперплоскость максимального зазора, что эквивалентно решению задачи квадратичного программирования

$$\min_{\omega, b} \|\omega\|^2 \quad \text{при условии} \quad y_i(\omega^\top x_i + b) \geq 1, \quad \forall i,$$

где $y_i \in \{-1, +1\}$ — метки двух рассматриваемых классов.

Для каждой пары (l, j) выполнялись следующие шаги:

1. Из исходной выборки извлекались все точки, принадлежащие классам l и j :

$$\{x_i : \text{label}(x_i) \in \{l, j\}\}.$$

2. Точкам назначались бинарные метки: -1 для класса l и $+1$ для класса j .
3. Задача методом опорных векторов решалась итерационным алгоритмом: вычислялась матрица $K = XX^\top$, затем множители Лагранжа α попарно обновлялись с учётом ограничений и пересчётом смещения b до сходимости.
4. По найденным α восстанавливался вектор нормали:

$$\omega = \sum_{i=1}^M \alpha_i y_i x_i,$$

а смещение b уточнялось по опорным векторам (точкам с $\alpha_i > 0$) по формуле усреднения $b \approx \text{mean}(y_i - \omega^\top x_i)$.

5. Полученные параметры ω и b сохранялись в массивы W и B с индексами (l, j) .

Построенные гиперплоскости образуют границы между критическими областями. Каждая область $\tilde{K}^l$ задаётся системой линейных неравенств

$$\tilde{K}^l = \{x \in \mathbb{R}^2 : \tilde{H}^l x \leq \tilde{g}^l\},$$

где строки матриц $\tilde{H}^l$ и $\tilde{g}^l$ соответствуют нормальям и смещениям гиперплоскостей, разделяющих класс l от его соседей.

2.4 Результаты визуализации

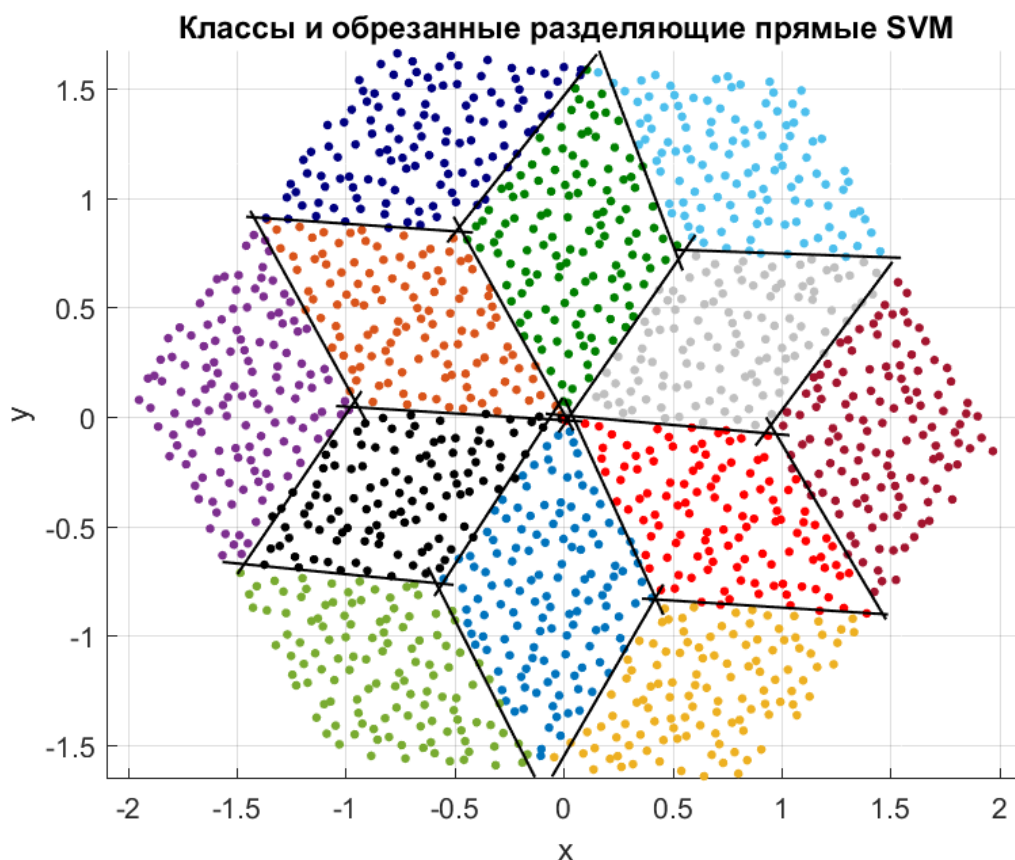


Рис. 2.1: Результаты синтеза субоптимальной обратной связи методом опорных векторов

На рис. 2.1 представлены результаты применения метода опорных векторов для синтеза субоптимальной обратной связи на основе данной выборки. График отображает: 18 чёрных линий, построенных методом SVM, которые определяют границы между соседними критическими областями. Каждая линия представляет оптимальную гиперплоскость для разделения двух классов с максимальным зазором, совокупность гиперплоскостей образует многоугольное разбиение двумерного пространства состояний на 12 выпуклых многогранников.

2.5 Характеристики и качество аппроксимации

Выбор линейного классификатора обоснован тем, что критические области в рассматриваемых задачах имеют многогранную структуру, а их

границы описываются гиперплоскостями. Поэтому использование линейного метода опорных векторов согласуется с теоретической моделью: разделение соседних областей выполняется гиперплоскостью максимального зазора.

В реализованном алгоритме обучение выполняется путём итерационного решения квадратичной задачи в двойственной форме с вычислением матрицы скалярных произведений $K = XX^T$ и последовательными обновлениями множителей Лагранжа α_i . Для приближения режима жесткого зазора применяется большое значение параметра C , что минимизирует количество нарушений зазора и приближает решение к постановке $y_i(\omega^T x_i + b) \geq 1$. При этом введён допуск `tol`, определяющий допустимую величину нарушения условий оптимальности, и критерий остановки по числу проходов без изменений `maxPasses`.

Качество аппроксимации границ критических областей оценивалось визуально по совпадению разделяющих прямых с «контактными» зонами между соседними классами, а также по устойчивости построения гиперплоскостей для всех выявленных пар соседства. В результате получено согласованное многогранное разбиение пространства состояний на области, соответствующие меткам исходной выборки.

2.6 Вычислительная сложность и время выполнения

Процесс синтеза субоптимальной обратной связи включал следующие этапы: чтение файла с готовой выборкой и извлечение координат и меток классов, для каждой пары соседних классов решение задачи квадратичного программирования — в среднем $O(n \cdot m_l^2)$ на пару, где $n = 2$ (размерность) и $m_l \approx 240$ (среднее число точек в двух классах вместе), отрисовка 1442 точек и 18 линий на двумерном графике.

В данной главе продемонстрирована практическая эффективность разработанного метода синтеза субоптимальных обратных связей на основе большой выборки данных (1442 точек, 12 классов, 18 пар соседства). Метод опорных векторов успешно выделил границы между критическими

областями, сохраняя их кусочно-аффинную структуру. Визуализация показывает четкую геометрическую структуру разбиения пространства состояний. Полученные результаты открывают возможность практического применения метода для синтеза управляющих законов в системах высокой размерности.

Заключение

В работе рассмотрена задача синтеза оптимальных систем на основе методов классификации. Предложен подход, объединяющий классическую теорию оптимального управления с современными методами машинного обучения.

Обоснована возможность применения методов линейной классификации для аппроксимации оптимальной обратной связи, опираясь на доказанную кусочно-аффинную структуру решения позиционных задач оптимального управления.

Разработан и формализован алгоритм поиска соседних критических областей на основе анализа расстояний между состояниями. Показано, что такой подход позволяет эффективно определить, между какими классами состояний необходимо строить разделяющие гиперплоскости.

Предложена методика применения линейного метода опорных векторов для построения аппроксимирующих гиперплоскостей. Обосновано, что использование линейного ядра обеспечивает соответствие структуре оптимального решения. Реализован полный программный комплекс, включающий модули для сбора и классификации данных, обучения моделей метода опорных векторов.

Проведены численные эксперименты на примере конкретной двумерной линейной системы, демонстрирующие практическую применимость предложенного метода. Результаты показали, что субоптимальная обратная связь обеспечивает качество управления, близкое к оптимальному, при значительном снижении вычислительных затрат.

Работа демонстрирует, что синтез традиционных подходов теории управления с современными методами анализа данных позволяет получить практически применимые решения для систем, где классические методы синтеза являются вычислительно неэффективными.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Дмитрук, Н.М. Синтез субоптимальных обратных связей в линейных задачах оптимального управления методом опорных векторов / Н.М. Дмитрук, М.А. Готовец // Известия Иркутского государственного университета. Серия Математика. – 2025. – Т. 39. – С. 1–15.
2. Габасов, Р. Методы оптимизации / Р. Габасов, Ф.М. Кириллова. – Минск: БГУ, 2011. – 472 с.

ПРИЛОЖЕНИЕ А

```
1 % Чтениеданных
data = readtable('classification.csv');

x = data.Var1;
y = data.Var2;
6 labels = data.Var3;

classes = unique(labels);
numClasses = numel(classes);

11 %% Палитрацветовдляклассов
colors = [
    0.00 0.45 0.74 % 1 синий
    0.85 0.33 0.10 % 2 оранжевый
    0.93 0.69 0.13 % 3 жёлтый
16 0.49 0.18 0.56 % 4 фиолетовый
    0.30 0.75 0.93 % 5 голубой
    0.47 0.67 0.19 % 6 зелёный
    0.64 0.08 0.18 % 7 бордовый
    0.00 0.00 0.00 % 8 чёрный
21 0.75 0.75 0.75 % 9 серый
    1.00 0.00 0.00 % 10 красный
    0.00 0.50 0.00 % 11 тёмнозелёный-
    0.00 0.00 0.50 % 12 тёмносиний-
];

26 %% Поисксоседнихклассов
R = 0.1; % радиуссоседства
neighbors = cell(numClasses,1); % соседикаждогокласса

31 for i = 1:numClasses-1
    ci = classes(i);
    idxi = (labels == ci);
    Xi = [x(idxi), y(idxi)];

36     for j = i+1:numClasses
        cj = classes(j);
        idxj = (labels == cj);
        Xj = [x(idxj), y(idxj)];

41         countPairs = 0;
```

```

46     for p = 1:size(Xi,1)
        dx = Xj(:,1) - Xi(p,1);
        dy = Xj(:,2) - Xi(p,2);
        dist = sqrt(dx.^2 + dy.^2);
        countPairs = countPairs + sum(dist <= R);

        if countPairs >= 5
            neighbors{i} = [neighbors{i}, cj];
51         neighbors{j} = [neighbors{j}, ci];
            break;
        end
    end
end
56 end

%% Построение гиперплоскостей SVM для всех пар соседних классов
W = cell(numClasses);
B = cell(numClasses);

61 for ii = 1:numClasses
    ci = classes(ii);
    neigh = unique(neighbors{ii});

66     for k = 1:numel(neigh)
        cj = neigh(k);
        jj = find(classes == cj);

        if ii >= jj || ~isempty(W{ii,jj})
71             continue;
        end

        idx_i = (labels == ci);
        idx_j = (labels == cj);

76         X_pair = [x(idx_i); x(idx_j)];
        Y_pair = [y(idx_i); y(idx_j)];
        X_train = [X_pair, Y_pair];

81         y_train = [-ones(sum(idx_i),1); +ones(sum(idx_j),1)];

        [M,n] = size(X_train);
86         y_train = y_train(:);

        % Для hard-margin формально C = inf,
        % численные итерации берут большое C

```

```

C = 1e6;
tol = 1e-3;
91 maxPasses = 10;

alpha = zeros(M,1);
b = 0;

96 % линейноеядро K = X*X'
K = X_train*X_train';

passes = 0;
while passes < maxPasses
101     numChanged = 0;

    for i = 1:M
        f_i = sum(alpha .* y_train .* K(:,i)) + b;
        E_i = f_i - y_train(i);

106     % Проверка KKT (soft-margin форма; при C=1e6 ~ hard-
        margin)
        if (y_train(i)*E_i < -tol && alpha(i) < C) || (
            y_train(i)*E_i > tol && alpha(i) > 0)

            % выбрать j != i
111            j = randi(M);
            while j == i
                j = randi(M);
            end

            f_j = sum(alpha .* y_train .* K(:,j)) + b;
            E_j = f_j - y_train(j);

            ai_old = alpha(i);
            aj_old = alpha(j);

121            % L,H
            if y_train(i) ~= y_train(j)
                L = max(0, aj_old - ai_old);
                H = min(C, C + aj_old - ai_old);
            else
126                L = max(0, ai_old + aj_old - C);
                H = min(C, ai_old + aj_old);
            end
            if L == H, continue; end

131            eta = 2*K(i,j) - K(i,i) - K(j,j);
            if eta >= 0, continue; end

```

```

136      % шаг по alpha_j
      alpha(j) = aj_old - y_train(j)*(E_i - E_j)/eta;

      % clip
      if alpha(j) > H, alpha(j) = H; end
      if alpha(j) < L, alpha(j) = L; end

141
      if abs(alpha(j) - aj_old) < 1e-12
          alpha(j) = aj_old;
          continue;
      end

146
      % alpha_i из условия sum alpha_i y_i = 0
      alpha(i) = ai_old + y_train(i)*y_train(j)*
          (aj_old - alpha(j));

      % пересчёт b
151      b1 = b - E_i ...
          - y_train(i)*(alpha(i)-ai_old)*K(i,i) ...
          - y_train(j)*(alpha(j)-aj_old)*K(i,j);

      b2 = b - E_j ...
156      - y_train(i)*(alpha(i)-ai_old)*K(i,j) ...
          - y_train(j)*(alpha(j)-aj_old)*K(j,j);

      if alpha(i) > 0 && alpha(i) < C
          b = b1;
161      elseif alpha(j) > 0 && alpha(j) < C
          b = b2;
      else
          b = 0.5*(b1 + b2);
      end

166
      numChanged = numChanged + 1;
  end
end

171
if numChanged == 0
    passes = passes + 1;
else
    passes = 0;
end

176
end

% восстановление w
w = X_train' * (alpha .* y_train);

```

```

181 % необязательно() уточнение b по опорным векторам , где alpha > 0
sv = find(alpha > 1e-6);
if ~isempty(sv)
    b = mean(y_train(sv) - X_train(sv,:)*w);
end

186
    W{ii,jj} = w;
    B{ii,jj} = b;
end
end

191 %% Визуализация: классы + разделяющие прямые SVM
figure; hold on; grid on;

for i = 1:numClasses
196     cl = classes(i);
    idx = (labels == cl);
    scatter(x(idx), y(idx), 10, colors(i,:), 'filled', ...
            'DisplayName', sprintf('Class %d', cl));
end

201 xlabel('x');
ylabel('y');
title('Классы и обрезанные разделяющие прямые SVM');
axis equal;

206 x_min = min(x); x_max = max(x);
y_min = min(y); y_max = max(y);

distThresh = 0.1;

211 for ii = 1:numClasses-1
    for jj = ii+1:numClasses
        if isempty(W{ii,jj})
            continue;
        end

216
        w = W{ii,jj};
        b = B{ii,jj};

        ci = classes(ii);
        cj = classes(jj);
221     idx_i = (labels == ci);
        idx_j = (labels == cj);
        X_pair = [x(idx_i); x(idx_j)];
        Y_pair = [y(idx_i); y(idx_j)];
226

```

```

231     if abs(w(2)) > 1e-8
        xx = linspace(x_min, x_max, 400);
        yy = -(w(1)*xx + b)/w(2);
    else
        xx = -b / w(1) * ones(1,400);
        yy = linspace(y_min, y_max, 400);
    end

    dmin = zeros(size(xx));
236   for k = 1:numel(xx)
        dx = X_pair - xx(k);
        dy = Y_pair - yy(k);
        dist = sqrt(dx.^2 + dy.^2);
        dmin(k) = min(dist);
241   end

    mask = dmin <= distThresh;
    xx(~mask) = NaN;
    yy(~mask) = NaN;
246

    plot(xx, yy, 'k-', 'LineWidth', 1);
end
end
251 legend('show', 'Location', 'eastoutside');
hold off;

```